IEEE MILCOM

IEEE ComSoc
IEEE Communications Society

A Lossy Compression Technique Using Disjoint Autoencoders for LoRa Communication

Santhosh Manoharan, Kang Gao and Subir Biswas

Networked Embedded and Wireless Systems (NeEWS) Laboratory,

Department of Electrical and Computer Engineering ,

Michigan State University, East Lansing, MI, USA

NeEWS
Networked Embedded & Wireless Systems Laboratory, Michigan State University



MICHIGAN STATE UNIVERSITY
Founded 1855

The IoT Revolution

IEEE MILCOM

IEEE ComSoc

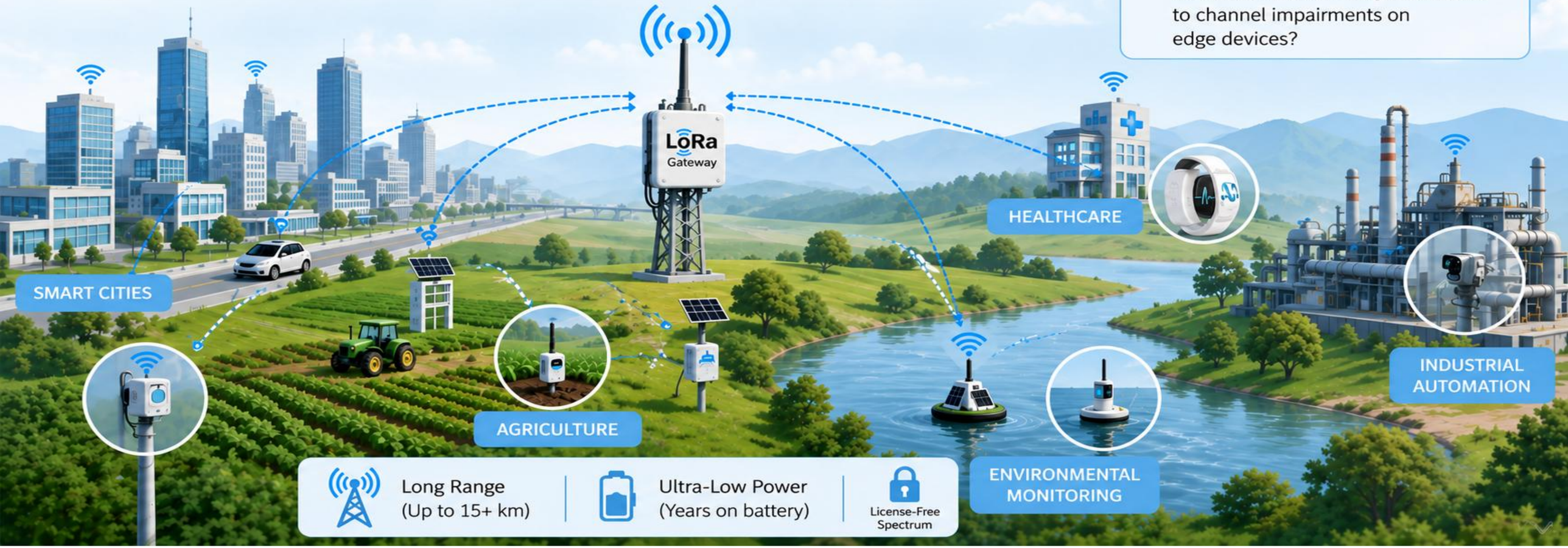
IoT deployments are **expanding** across smart cities, agriculture, healthcare, environmental monitoring, and industrial automation.

Sensor nodes need to **communicate over large areas** with minimal power, driving demand for LPWANs.

LoRa has emerged as the **leading** LPWAN technology: long-range, ultra-low power, license-free spectrum.

RESEARCH QUESTION

How do we compress images for LoRa while maintaining robustness to channel impairments on edge devices?



Why Are Images over LoRa Hard?

22 bps – 27 kbps

LoRa data rate range

51 bytes

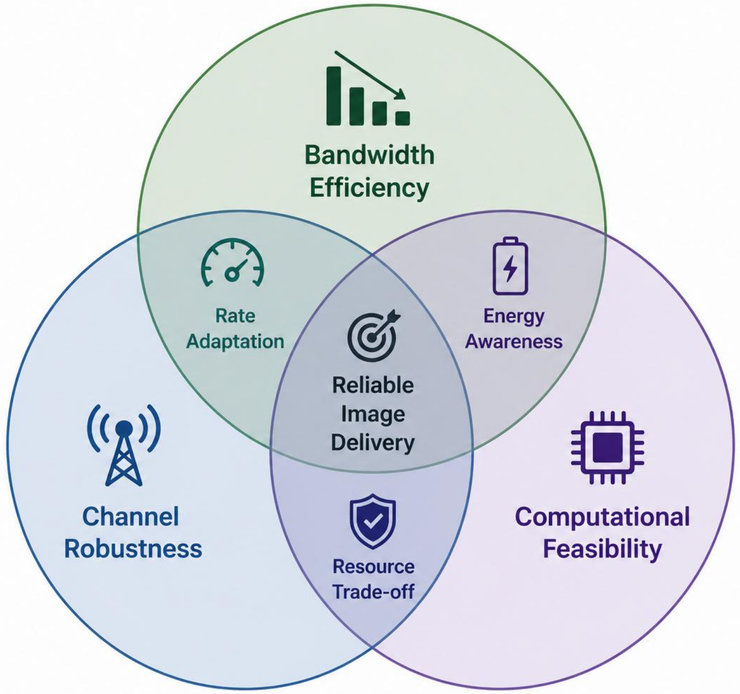
Max payload in the most robust settings

−20 to +10 dB

Typical operating SNR

64.54 s

To send ONE raw 32×32×3 image on our testbed



Sending raw images over LoRa is impractical, so an efficient compression mechanism is essential.
Can existing techniques cut bandwidth, survive the channel, and run on the edge?

Gap in Existing Approaches

Approach	Bandwidth reduction	Channel robustness	Edge feasible	Real LoRa validation
Classical codecs (JPEG, JPEG 2000, JPEG-LS/XR)	Yes	No	Yes	No
PCA	Yes	No	Yes	No
Standard autoencoders	Yes	No	Yes	No
Channel-aware AE / Deep JSCC	Yes	Partial	No	No
This work: disjoint AE + learnable LoRa channel	Yes	Yes	Yes	Yes

Why prior robust methods fall short on LoRa: they train against analytical AWGN / Rayleigh models that miss the combined effect of quantization, CSS modulation, and symbol-level errors, and they add compute that edge nodes cannot afford.

Contributions

C1

Disjoint Autoencoder

Symmetric CNN encoder/decoder trained jointly, deployed separately on transmitter and receiver. Latent dimension n × l tunes bandwidth, quality, and complexity.

C2

Learnable LoRa Channel

Differentiable approximation of CSS modulation, channel impairment, and demodulation, used only during training to make latent robust to real LoRa distortion.

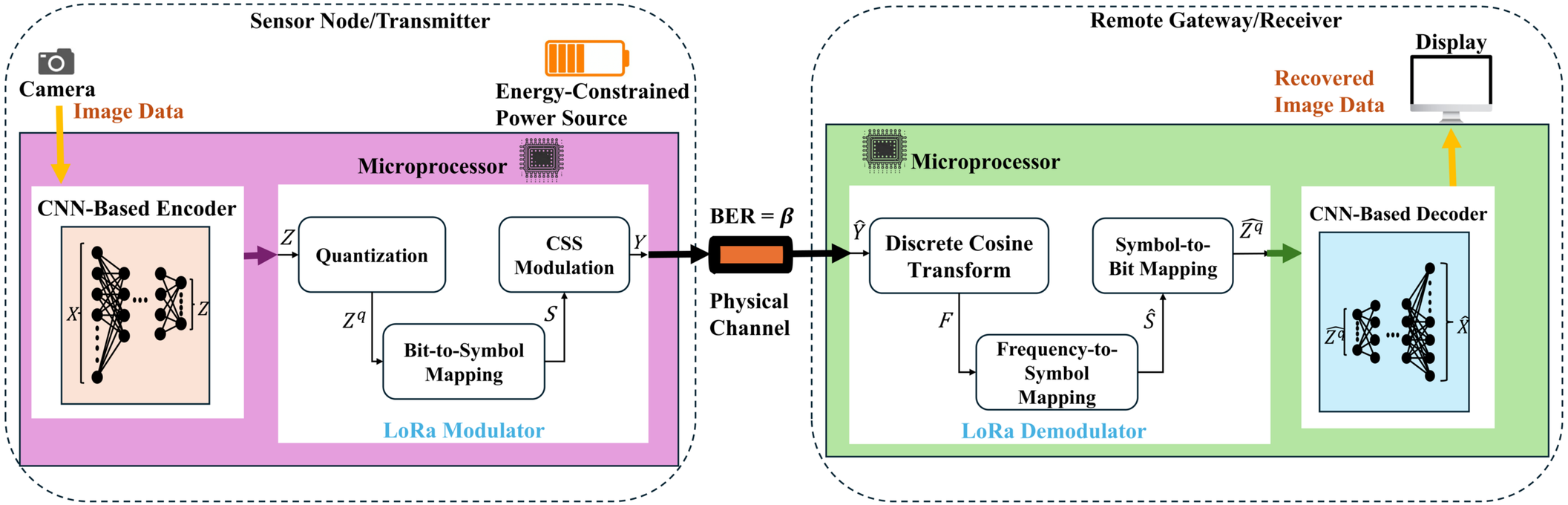
C3

Real-Hardware Trade-offs

Systematic analysis of bandwidth, reconstruction quality, distance, latency, and energy on Raspberry Pi Zero 2W over a live 915 MHz LoRa link.

Validated end-to-end: Raspberry Pi Zero 2W · 915 MHz LoRa · CRC / FEC off · 10 cm – 100 m outdoor LoS

System Model



- **Architecture:** Asymmetric two-node system with a constrained edge transmitter and a resource-rich gateway receiver.
- **Processing pipeline:**

  CNN Encoder → Quantization → LoRa Modulation → Wireless Channel → Demodulation → CNN Decoder.
- **Edge constraints:** Jointly limited by energy budget, computational capacity, and narrow-band communication under stochastic channel impairments.

Disjoint Autoencoder

Module	Layer	Output shape
Encoder	Conv2D + ReLU	$\boldsymbol{p \times q \times n}$
	MaxPooling2D	$\boldsymbol{\frac{p}{2} \times \frac{q}{2} \times n}$
	Conv2D + ReLU	$\boldsymbol{\frac{p}{2} \times \frac{q}{2} \times n}$
	Reshape	$\boldsymbol{n \times \frac{pq}{4}}$
	Dense + ReLU	$\boldsymbol{n \times l}$ **(latent Z)**
Decoder	Dense + ReLU	n × pq/4
	Reshape	p/2 × q/2 × n
	Conv2D + ReLU	p/2 × q/2 × n
	UpSampling2D	p × q × n
	Conv2D + ReLU	p × q × c

Encoder:

- Input: image X of dimension p × q × c (32×32×3 for CIFAR-10).
- Output: compact latent representation $Z \in R^{n \times l}$.
  - n = number of convolutional filters (architectural parameter)
  - l = compressed latent dimension (architectural parameter).

Decoder:

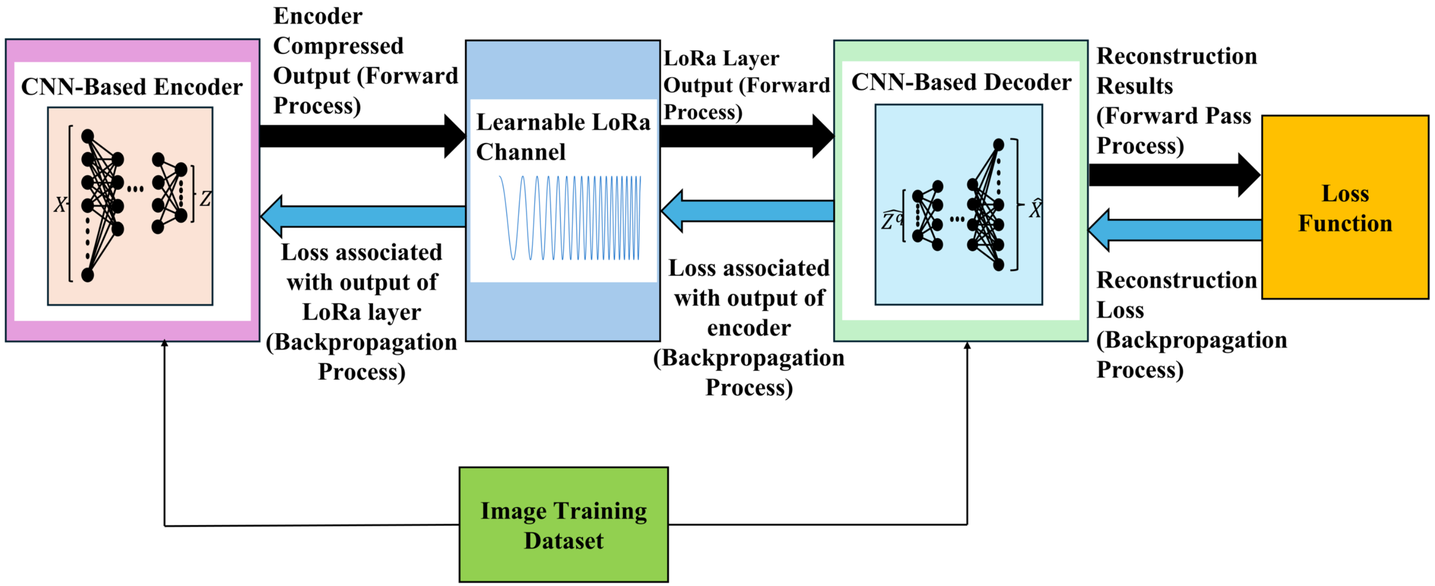
- Input: compact latent representation $Z \in R^{n \times l}$.
- Output: Reconstructed image X of dimension p × q × c.
- Mirrors the encoder in reverse: 5 successive layers progressively expanding latent to original image dimensions.

Bandwidth reduction

$$BR = 1 - \frac{n \times l}{p \times q \times c}$$

- **BR range:** 99.48% (n=4, l=4) → 0% (n=64, l=48)
- **Primary evaluation point:** n=32, l=24 (75% BR)

Two-Stage Offline Training



Stage 1: Pre-train

- Encoder E_θ and decoder $D_{\theta'}$ are jointly trained for 50 epochs without the channel module.
- Pre-trained weights θ and θ' saved at end of Stage 1 as initialization for Stage 2.

Stage 2: Fine-tune with channel

- Learnable LoRa channel module M is inserted between encoder and decoder.
- Full pipeline $E_\theta \rightarrow \mathrm{M} \rightarrow D_{\theta'}$, fine-tuned end-to-end for 30 additional epochs.
- Stage 2 is repeated for each SNR training level to create a family of models.

Loss: MAE

$$L(X,\hat{X}) = \frac{1}{p \times q \times c}|X - \hat{X}|$$

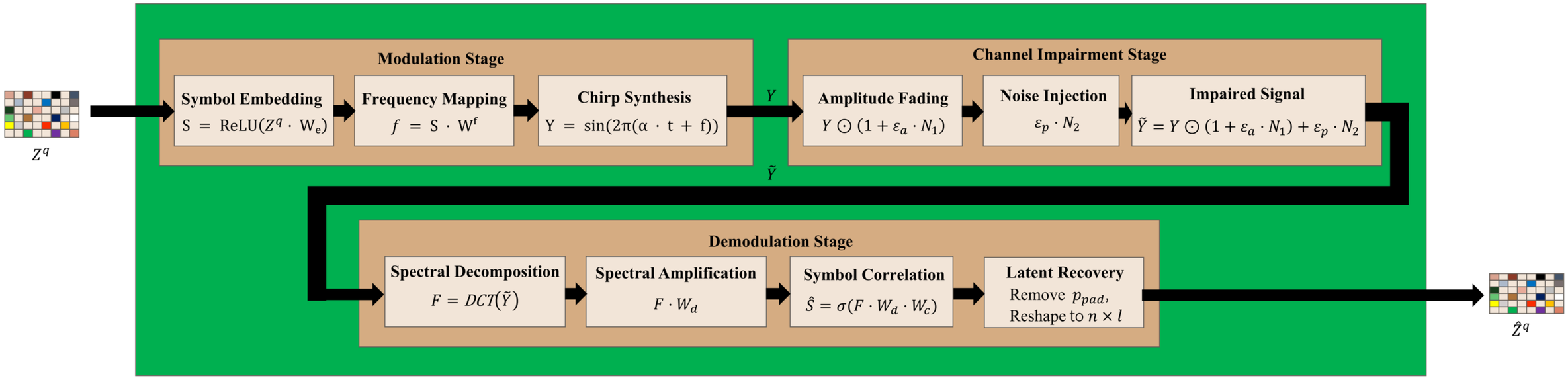
Straight-through estimator

Rounding treated as identity in backpropagation so gradients reach E_θ

Deploy disjointly

- Encoder weights → Transmitter
- Decoder weights → Receiver

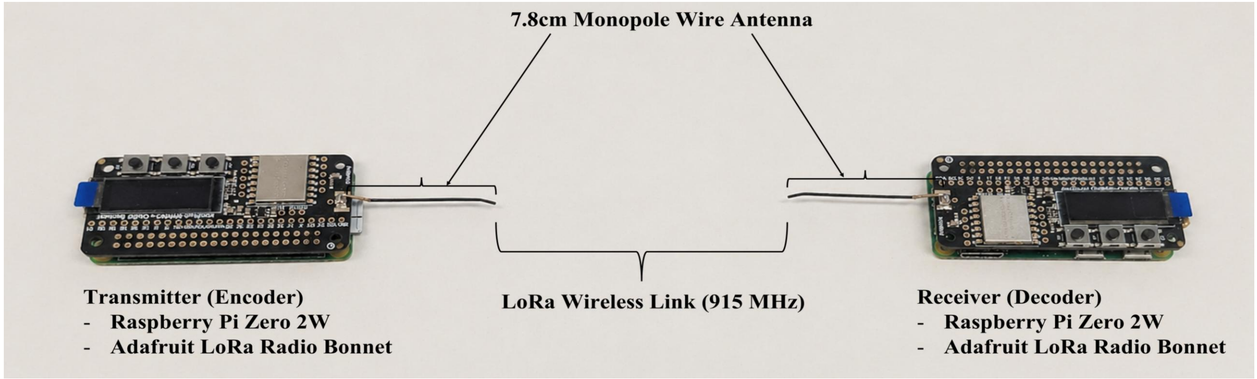
Learnable LoRa Channel Module



Three fully differentiable learnable stages approximating:
(1) **Modulation**, (2) **Channel impairment**, (3) **Demodulation**.
All parameters jointly optimized with encoder and decoder during Stage 2 fine-tuning.

1) **Modulation**: embedding matrix $W_e \in \mathrm{R}^{SF \times 2^{SF}}$ projects SF-bit groups to frequency feature space. Trainable chirp slope α initialized at BW/2 = 62,500 Hz .
2) **Channel Impairment**: $\tilde{Y} = Y \odot (1 + \varepsilon_a \cdot N_1) + \varepsilon_p \cdot N_2$,
3) **Demodulation**: DCT-based frequency analysis + learnable decoding matrices $W_d \in \mathrm{R}^{2^{SF} \times 2^{SF}}$ and $W_c \in \mathrm{R}^{2^{SF} \times SF}$ to give Z^q.

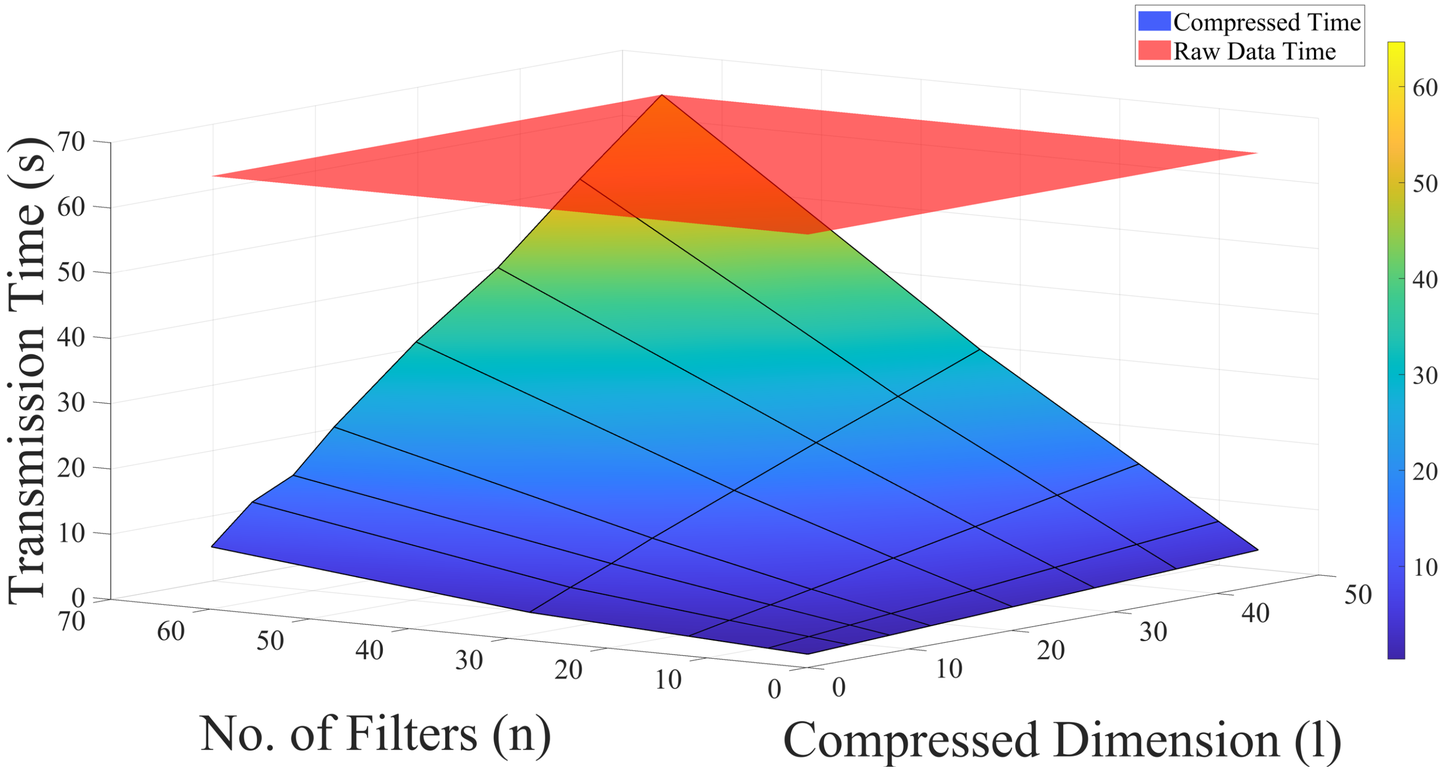
Experimental Setup





Parameter	Value
Platform	Raspberry Pi Zero 2W (quad-core Cortex-A53, 512 MB)
Radio / antenna	Adafruit LoRa Radio Bonnet, 915 MHz; 7.8 cm monopole
LoRa PHY	BW 500 kHz, SF 7, CR 4/5, 23 dBm
Payload	250 bytes per packet
Error control	CRC and FEC disabled
Dataset	CIFAR-10 test set (10,000 images)
Field test	Outdoor LoS, 10 cm – 100 m, 1 m steps, 250 images/point
Baselines	AE without channel, JPEG-LS, JPEG-XR, PCA
Power	10 Ω shunt, 5 V supply, oscilloscope @ 1470 Hz

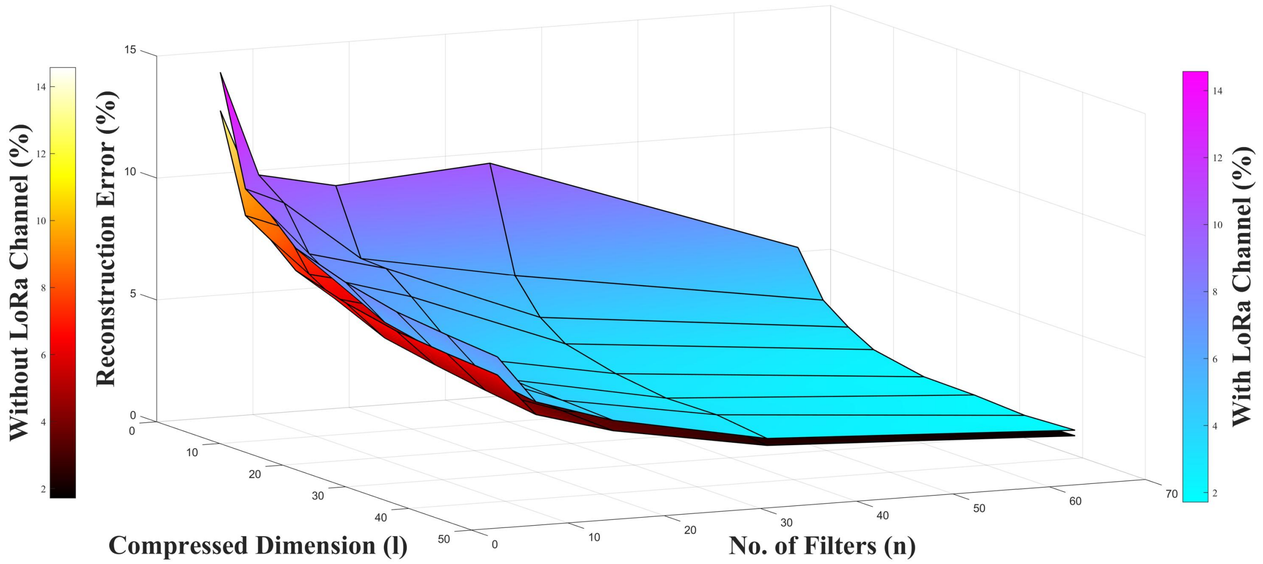
Bandwidth & Transmission Time



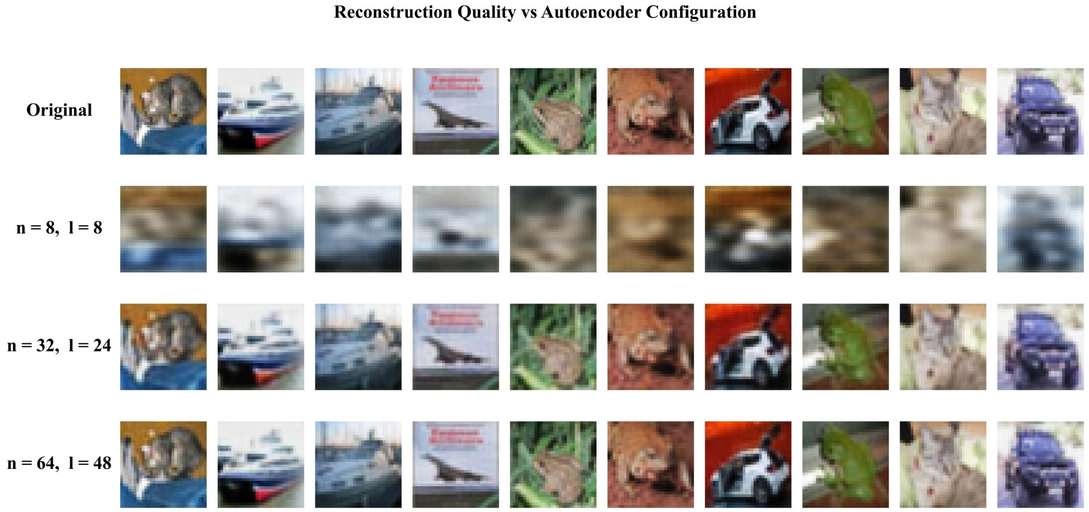
Measured transmission time per image at 10 cm (BW 500 kHz, SF 7)

- Bandwidth reduction surface is monotonically decreasing in both n and l directions.
- Surface is smooth with no discontinuities; bandwidth reduction behaves predictably and continuously.
- Intermediate configurations can be selected with confidence to meet specific bandwidth constraints.
- Raw Transmission : 64.54 s
  - Uncompressed 32×32×3 CIFAR image
- n=32, l=24 : 15.47 s
  - 75% reduction in transmission time. Achieves practical near-real-time performance
- n=4, l=4 : 0.31 s
  - 99.5% bandwidth reduction, Full image transmitted in under one second

Reconstruction Quality (Noise-Free)



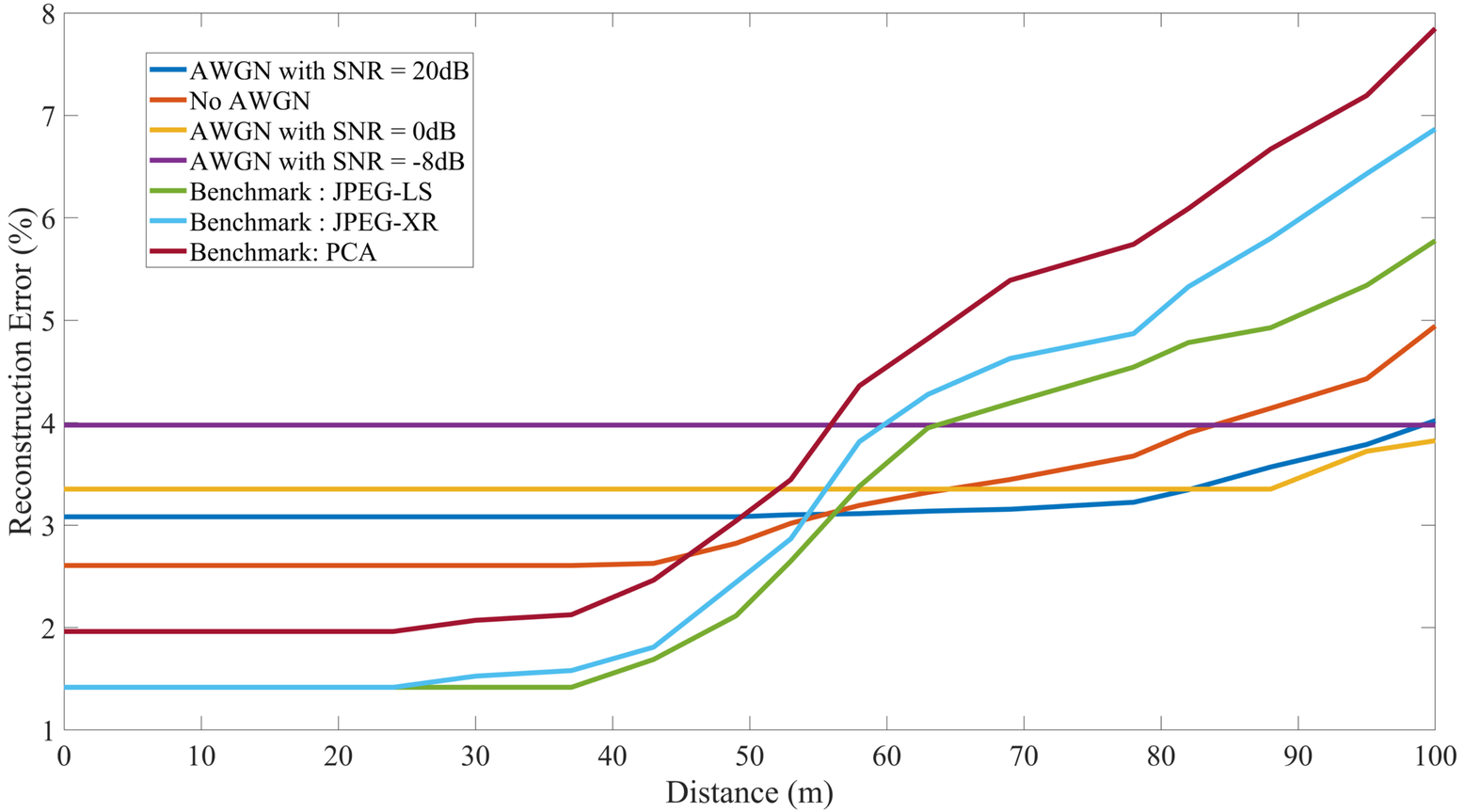
Reconstruction error vs. latent dimension (with / without LoRa layer)



Original vs. n=8,l=8 · n=32,l=24 · n=64,l=48

- Both surfaces (with and without learnable channel) show reconstruction error decreasing monotonically as n and l increase.
- Without channel module: error ranges 1.71%-2.80% at n≥32, l≥24.
- With learnable channel module: error ranges 2.01%-3.14% at n≥32, l≥24.
- Overhead: ~0.3-0.34 %, modest, bounded, and consistent across full parameter space.

Reconstruction Error vs. Distance



Field measurements, n = 32, l = 24; legend SNR = training SNR of the learnable channel

- **Short range (≤ 24 m)**

  Classical codecs lead: JPEG-XR / JPEG-LS 1.42%, PCA 1.96%

- **At 100 m, baselines collapse**

  JPEG-XR 6.87% · JPEG-LS 5.78% · PCA 7.85% · AE w/o channel 4.94%

- **Channel-trained models**

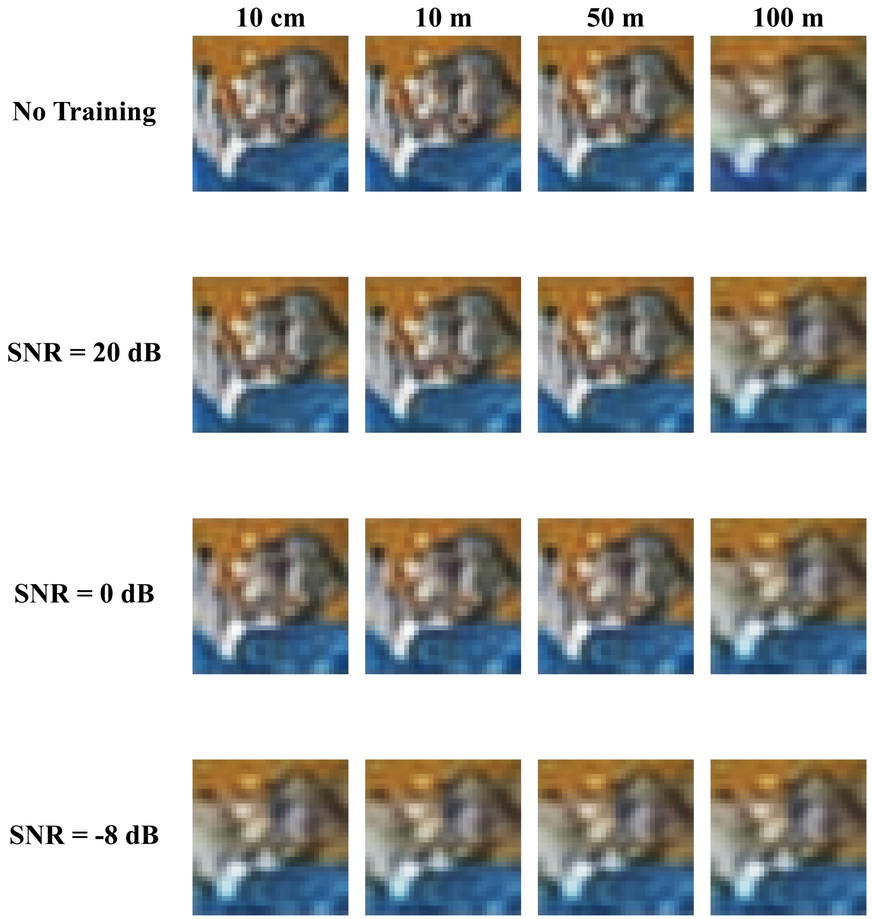
  SNR 20 dB: 3.22% to 78 m → 4.02%
  SNR 0 dB: 3.35% to 88 m → 3.82%

- **SNR −8 dB model**

  Near-constant 3.98% across the full 0–100 m range

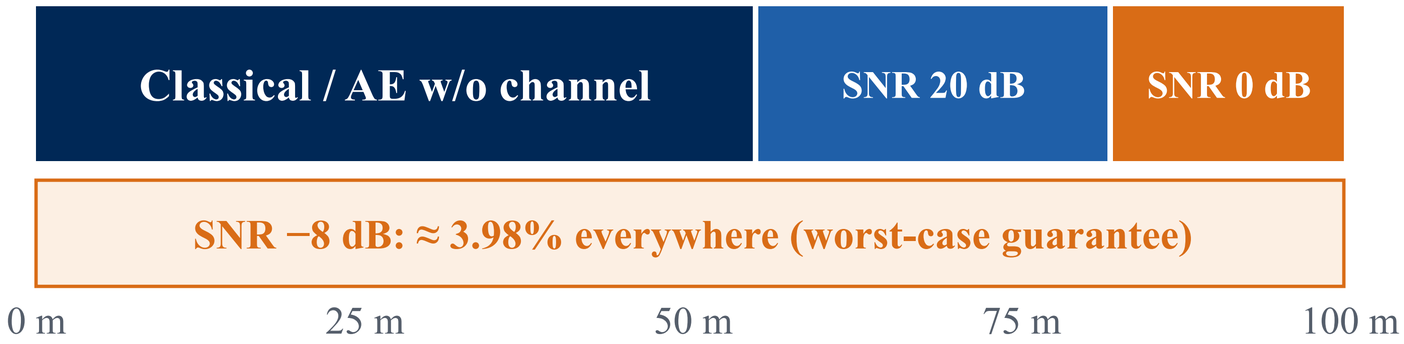
Distance Decides the Right Model

Reconstruction Quality vs Training Configuration and Distance (Image 0)



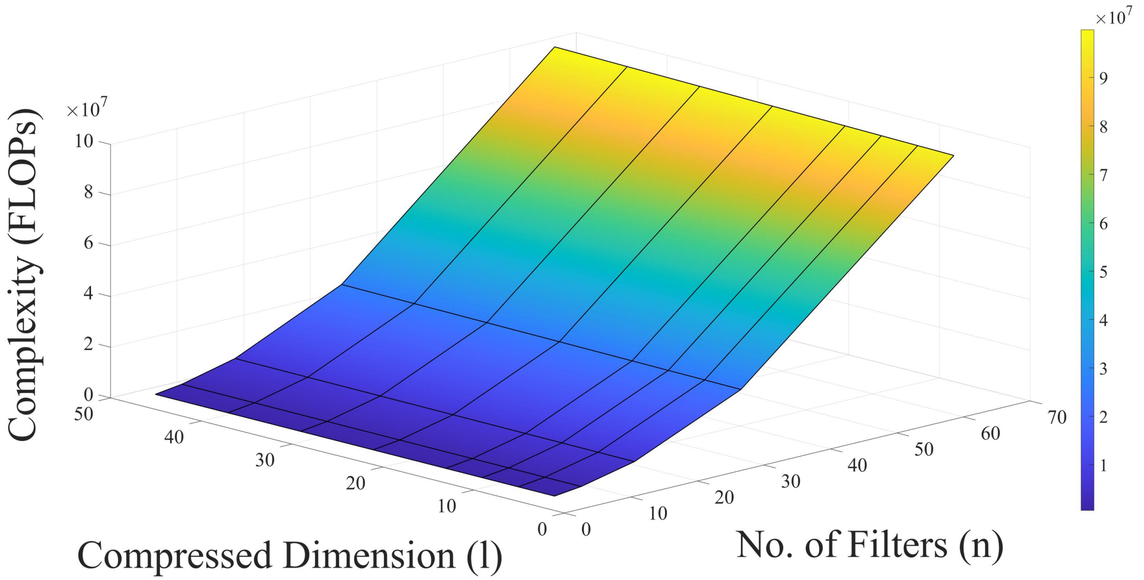
Received images at 10 cm, 10 m, 50 m, 100 m

Lowest-error choice by distance

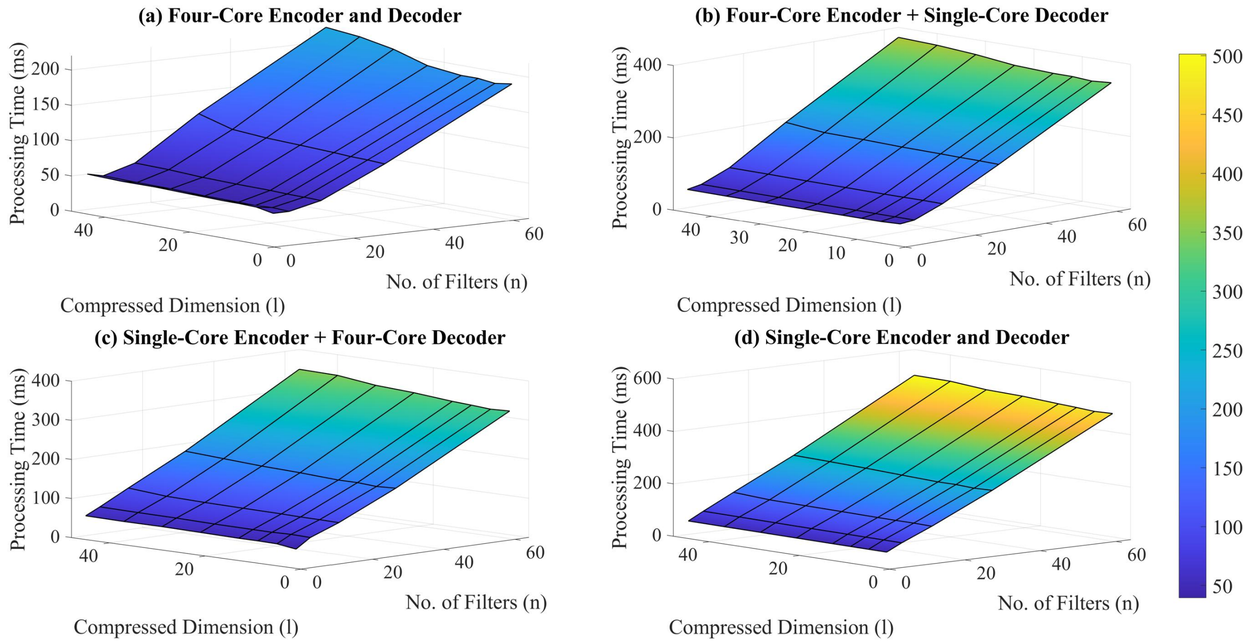


- **No single configuration dominates** every axis
- **Harsher training noise** extends the usable range at a small cost in short-range quality
- **Deployment rule:** estimate typical link distance / receiver SNR, then load the matching encoder–decoder pair

Complexity & Processing Time



FLOPs vs. latent dimension

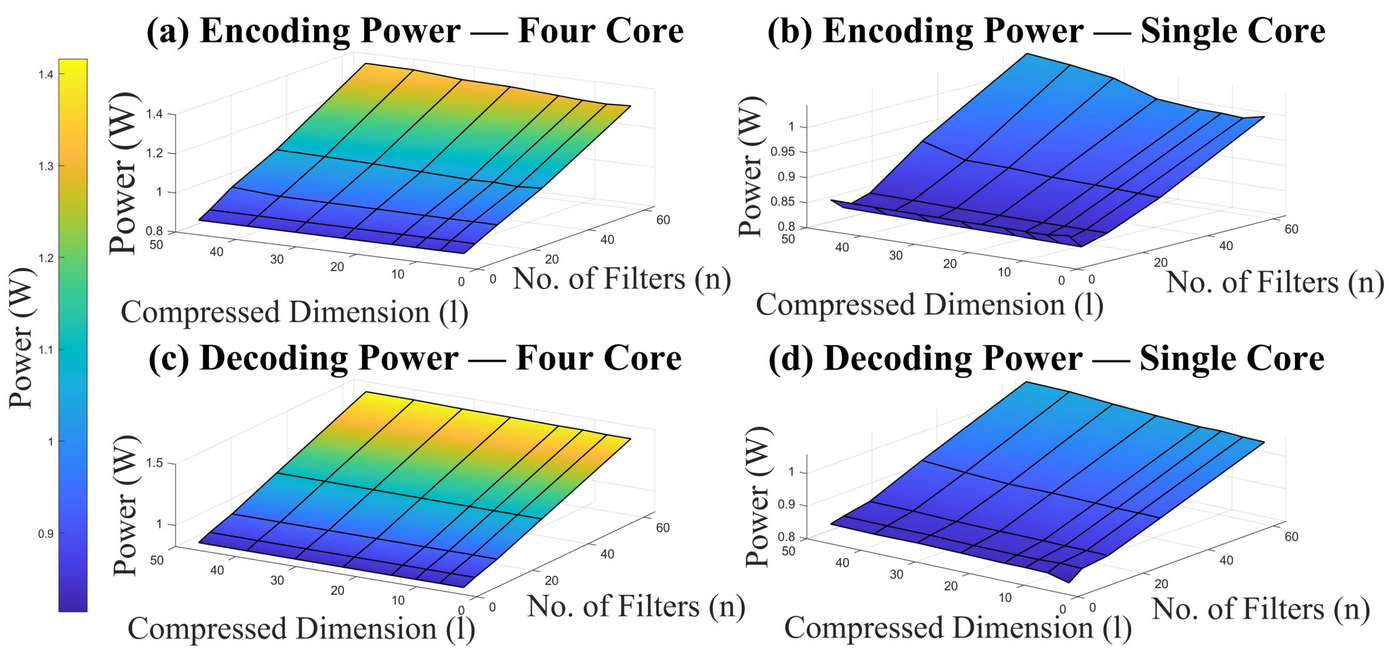


Processing Time vs latent dimension

- FLOPs range: ~500,000 at (n=4, l=4) to >100 million at (n=64, l=48).
- Complexity scales quadratically with n and linearly with l.
- Smooth, predictable scaling confirms framework complexity can be estimated from architectural parameters alone.

- Processing Time mimics the trend observed in complexity
- Strategy (a) consistently the fastest and strategy (d) is the slowest.
- Strategy (b) vs (c):
  - (b) better for n<16 (decoder lightweight);
  - (c) better for n≥32 (encoder heavier).

Power & Energy on the Edge Device



Encoding / decoding power vs. latent dimension, four-core vs. single-core

	Single-core	Four-core
Instantaneous power	0.82–1.06 W	0.82–1.42 W
Total energy	39–526.29 J	**33.65–303.66 J**

Power scales with n, largely insensitive to l

Lower power ≠ lower energy: single-core runs longer and spends more total energy

Use single-core only when cores must be freed for concurrent tasks

Conclusions

1

Robust by design

A differentiable LoRa channel in the training loop yields latents that survive real CSS transmission without CRC or FEC.

2

Distance picks the model

Each training SNR defines an operating region; −8 dB gives ≈ 3.98% error flat to 100 m vs. up to 7.85% for baselines.

3

Latent size picks the budget

n × l trades bandwidth (up to 99.48% BR) against quality; n dominates compute and power.

4

Cores pick latency vs. energy

Quad-core execution is fastest and uses the least total energy on Pi Zero 2W.

Future work

- Domain-specific IoT imagery (agriculture, environmental monitoring, surveillance) beyond CIFAR-10
- Non-line-of-sight, urban multipath, and vegetation; channel module informed by measured LoRa responses
- Runtime model selection: switch among SNR-trained models based on measured receiver SNR

Thank You

Santhosh Manoharan - manoha10@msu.edu

Network Embedded and Wireless Systems (NeEWS) Lab, Michigan State University

NeEWS
Networked Embedded & Wireless Systems Laboratory, Michigan State University